

20 questions pandas

Diagnostic de niveau /20



Avant ton prochain code review. Durée conseillée : 20 min · 6 paliers · corrigé détaillé.

Objectif : mesurer tes réflexes pandas réels, pas seulement ta capacité à faire tourner un notebook. Ce test vérifie si tu sais quand un `groupby` te trompe sur l'ordre, quand un `merge` duplique tes lignes en silence et pourquoi une assignation sur un sous-DataFrame peut ne rien faire du tout.

Repères : 10/20 = bases posées, 15/20 = junior solide, 18/20 = profil qu'on garde sur les jointures et l'indexation.

Candidat _____ Date _____ Note _____ /20

Observation terrain : la plupart des DataFrames pandas « qui tournent » sans erreur cachent quand même un biais silencieux, un doublon non détecté, un ordre de tri qui change d'une exécution à l'autre. Ce test cible ces angles morts, palier par palier, du chargement du fichier jusqu'aux pièges d'indexation.

Palier 1 : premiers réflexes.

Trois questions sur la lecture et l'inspection d'un fichier avant de toucher à quoi que ce soit.

Question 1. Tu reçois un DataFrame et tu veux voir en un coup d'oeil les types de chaque colonne et le nombre de valeurs non manquantes. Tu appelles :

- a) `df.info()`
- b) `df.describe()`
- c) `df.head()`
- d) `df.columns`

Question 2. Un collègue t'envoie un export FR où les colonnes sont séparées par des point-virgules (Nom;Ville;Montant). Tu charges le fichier avec `pd.read_csv('export.csv')` sans autre paramètre. Que se passe-t-il ?

- a) Rien, pandas détecte automatiquement le séparateur.
- b) Le fichier se charge, mais en une seule colonne : pandas suppose une virgule par défaut.
- c) Une erreur `FileNotFoundError`.
- d) Les nombres sont interprétés comme du texte.

Question 3. Pour compter le nombre de valeurs manquantes par colonne, la façon la plus directe est :

- a) `df.dropna().count()`
- b) `df.fillna(0).sum()`
- c) `df.isna().sum()`
- d) `df.count(axis=1)`

Palier 2 : nettoyage.

Quatre questions sur les NaN, les types de données et les doublons, la base d'un jeu de données fiable.

Question 4. Tu veux supprimer les lignes où la colonne `email` est manquante, mais garder les lignes qui ont un NaN ailleurs (par exemple dans `telephone`). Tu écris :

- a) `df.dropna()`
- b) `df.dropna(how='all')`
- c) `df.dropna(axis=1)`
- d) `df.dropna(subset=['email'])`

Question 5. Une colonne `montant` contient du texte avec virgule décimale française, par exemple `'1 234,56'`. Tu écris :

```
df['montant'] = df['montant'].str.replace(',', '.').astype(float)
```

Que se passe-t-il à cause de l'espace utilisé comme séparateur de milliers ?

- a) La conversion échoue avec une `ValueError`, l'espace n'étant pas un caractère numérique valide.
- b) pandas ignore automatiquement les espaces.
- c) Le résultat vaut 1234.56, l'espace est retiré par `astype`.
- d) La colonne devient un mélange de texte et de nombres sans erreur.

Question 6. Pour savoir si un `DataFrame` contient des lignes strictement dupliquées, sans encore les supprimer, tu utilises :

- a) `df.unique()`
- b) `df.duplicated().sum()`
- c) `df.nunique()`
- d) `df.drop_duplicates().count()`

Question 7. Après `df.drop_duplicates()`, sur un groupe de lignes identiques, quelle occurrence pandas garde-t-il par défaut ?

- a) La dernière ligne rencontrée.
- b) Une ligne choisie aléatoirement.
- c) La première ligne rencontrée.
- d) Aucune : toutes les lignes du groupe sont supprimées.

Palier 3 : groupby et agrégations.

Quatre questions sur ce qui se passe vraiment quand tu regroupes des lignes.

Question 8. Tu veux, pour chaque ville, la moyenne du montant et le nombre de commandes, en une seule opération. Tu écris :

- a) `df.groupby('ville')['montant'].mean()`
- b) `df.groupby('ville').agg(moyenne=('montant', 'mean'), nb=('montant', 'count'))`
- c) `df.groupby('ville').mean()`
- d) `df.pivot_table('ville')`

Question 9. Après `df.groupby('ville')['montant'].sum()`, le résultat a `ville` comme index et non comme colonne classique, ce qui complique un merge ultérieur. Pour repasser `ville` en colonne normale, tu ajoutes :

- a) `.reset_index()`
- b) `.set_index('ville')`
- c) `.to_frame()`
- d) `.sort_index()`

Question 10. Tu veux ajouter une colonne `moyenne_ville` qui répète, pour chaque ligne, la moyenne du montant de sa ville (même nombre de lignes qu'au départ). `agg()` ne convient pas, car il réduit le nombre de lignes. Tu utilises :

- a) `df.groupby('ville')['montant'].sum()`
- b) `df.groupby('ville')['montant'].transform('mean')`
- c) `df.groupby('ville')['montant'].agg('mean')`
- d) `df['montant'].mean()`

Question 11. Tu compares `df['ville'].value_counts()` et `df.groupby('ville').size()`. Quelle différence pratique change l'ordre du résultat ?

- a) Les deux trient toujours par ordre alphabétique des villes.
- b) `groupby().size()` trie toujours par fréquence décroissante, comme `value_counts()`.
- c) `value_counts()` trie par fréquence décroissante; `groupby().size()` suit l'ordre des clés groupées (généralement alphabétique).
- d) `value_counts()` ignore les valeurs manquantes, `groupby().size()` les inclut toujours de la même façon.

Palier 4 : jointures et merge.

Quatre questions sur les pièges les plus fréquents des entretiens techniques : un merge qui duplique ou qui vide silencieusement une table.

Question 12. Tu joins clients et commandes avec `pd.merge(clients, commandes, on='client_id', how='left')`. Un client n'a jamais commandé. Que devient sa ligne ?

- a) Elle disparaît du résultat.
- b) Elle provoque une erreur de clé manquante.
- c) Elle est dupliquée deux fois.
- d) Elle reste dans le résultat, avec les colonnes de commandes à NaN.

Question 13. Pour diagnostiquer, après un merge, quelles lignes viennent seulement de la table de gauche, seulement de la droite ou des deux, tu ajoutes le paramètre :

- a) `indicator=True`
- b) `validate='1:1'`
- c) `suffixes=('_g', '_d')`
- d) `how='outer'`

Question 14. Tu as deux DataFrames avec les mêmes colonnes (`ventes_janvier` et `ventes_fevrier`) et tu veux simplement les empiler l'un sous l'autre, sans aligner de clé commune. L'outil adapté est :

- a) `pd.merge(ventes_janvier, ventes_fevrier)`
- b) `pd.concat([ventes_janvier, ventes_fevrier])`
- c) `ventes_janvier.join(ventes_fevrier)`
- d) `pd.merge_ordered(ventes_janvier, ventes_fevrier)`

Question 15. `clients` et `commandes` ont toutes les deux une colonne `date`. Après `pd.merge(clients, commandes, on='client_id')`, tu retrouves `date_x` et `date_y` dans le résultat. Pourquoi ?

- a) pandas a fusionné les deux colonnes `date` en une seule par erreur.
- b) Le merge a échoué silencieusement.
- c) pandas suffixe automatiquement les colonnes homonymes non utilisées comme clé, avec `_x` pour la gauche et `_y` pour la droite.
- d) C'est un bug de pandas à corriger avec `.rename()`.

Palier 5 : indexation et filtres.

Trois questions sur les erreurs de syntaxe qui plantent un notebook ou qui filtrent silencieusement le mauvais sous-ensemble.

Question 16. Pour sélectionner une ligne par sa position (la 3^e ligne du DataFrame, peu importe la valeur de son index), tu utilises :

- a) `df.loc[3]`
- b) `df.at[3]`
- c) `df.get(3)`
- d) `df.iloc[2]`

Question 17. Un collègue junior écrit `df[df['ville'] == 'Lyon' and df['montant'] > 100]` pour filtrer les commandes lyonnaises de plus de 100 euros. Le code plante. Pourquoi ?

- a) Sur des Series pandas, il faut remplacer `and` par `&` et mettre chaque condition entre parenthèses : `(df['ville']=='Lyon') & (df['montant']>100)`.
- b) Il faut inverser l'ordre des deux conditions.
- c) Il faut utiliser `or` à la place de `and`.
- d) Le problème vient du nom de colonne `montant`, trop court.

Question 18. Tu veux garder les commandes dont la ville est Lyon, Paris ou Nice. Plutôt que d'empiler trois conditions avec `|` (OR), l'écriture la plus lisible et la plus sûre est :

- a) `df[(df['ville']=='Lyon') | (df['ville']=='Paris') | (df['ville']=='Nice')]`
- b) `df[df['ville'].isin(['Lyon','Paris','Nice'])]`
- c) `df[df['ville'] in ['Lyon','Paris','Nice']]`
- d) `df.query('ville == Lyon or Paris or Nice')`

Palier 6 : les pièges qui trahissent un junior.

Deux questions, celles qui reviennent le plus souvent en entretien technique pandas, parce qu'elles font la différence entre lire du code et le comprendre.

Question 19. Tu écris `sub = df[df['montant']>100]` puis `sub['statut'] = 'ok'`. pandas t'avertit avec un `SettingWithCopyWarning`. La correction robuste est :

- a) Ignorer l'avertissement, le résultat est de toute façon correct.
- b) Renommer la variable `sub` en `sub2`.
- c) Créer `sub` avec un `.copy()` explicite, ou écrire directement `df.loc[df['montant']>100, 'statut'] = 'ok'` sur le DataFrame d'origine.
- d) Remplacer `sub['statut'] = 'ok'` par `sub.statut = 'ok'`.

Question 20. Pour supprimer définitivement la colonne `code_postal` d'un DataFrame `df`, l'écriture correcte est :

- a) `df.drop('code_postal')`
- b) `df['code_postal'] = None`
- c) `df.dropna('code_postal')`
- d) `df.drop(columns=['code_postal'], inplace=True)`

Corrigé, paliers 1 à 3.

Un point par bonne réponse. Le concept d'abord, puis l'erreur type que fait un junior sur cette question précise.

- 1. a)** `info()` affiche dtypes, nombre de valeurs non nulles et mémoire par colonne. Le piège type junior : confondre avec `describe()`, qui ne porte que sur les colonnes numériques et ignore les types.
- 2. b)** Sans `sep=';'`, `read_csv` suppose une virgule et charge tout dans une seule colonne texte. Le fichier semble se charger sans erreur, mais il n'a qu'une colonne : ce piège passe souvent inaperçu jusqu'au premier `groupby`.
- 3. c)** `isna()` renvoie un masque booléen colonne par colonne, et `sum()` additionne les True. Le réflexe junior fréquent, `dropna().count()`, ne donne que le total de lignes restantes, pas le détail par colonne.
- 4. d)** `subset` limite la vérification des NaN aux colonnes précisées. Sans lui, `dropna()` supprime toute ligne contenant un seul NaN, même dans une colonne qui n'intéresse pas l'analyse : le jeu de données fond sans prévenir.
- 5. a)** `str.replace(',', '.')` gère la virgule décimale mais pas l'espace comme séparateur de milliers : `astype(float)` échoue avec une `ValueError`. Le piège type junior : tester sur un échantillon propre, sans espace, et découvrir l'erreur seulement en production.
- 6. b)** `duplicated()` marque True sur chaque occurrence redondante, `sum()` additionne. Un junior confond souvent avec `nunique()`, qui compte les valeurs distinctes, pas les doublons.
- 7. c)** `keep='first'` est le comportement par défaut de `drop_duplicates()` : la première occurrence rencontrée dans l'ordre du DataFrame est conservée. Ne pas le vérifier peut faire disparaître la bonne ligne (la plus récente, par exemple) si le fichier n'est pas trié comme prévu.
- 8. b)** `agg()` avec des tuples nommés calcule plusieurs agrégations en une passe et nomme directement les colonnes. Le réflexe junior : enchaîner plusieurs `.groupby().mean()` séparés, plus lent et plus dur à relire.
- 9. a)** Après un `groupby`, la clé de regroupement devient l'index du résultat. `reset_index()` la repasse en colonne normale, indispensable avant un merge ou un export propre. L'oublier casse souvent le merge suivant sur la mauvaise clé.
- 10. b)** `transform()` renvoie une série de même longueur que le DataFrame d'origine, alignée ligne à ligne; `agg()` réduit à une ligne par groupe. Le piège junior : utiliser `agg()` puis tenter un merge manuel pour rattacher la moyenne à chaque ligne, alors que `transform()` le fait directement.
- 11. c)** `value_counts()` trie toujours par fréquence décroissante; `groupby().size()` conserve l'ordre des clés groupées. Comparer les deux sans y penser fait croire à un changement de résultat, alors que seul l'ordre d'affichage diffère.

Corrigé, paliers 4 à 6.

Jointures, indexation, pièges de junior : la partie qui fait vraiment la différence en entretien technique.

- 12. d)** Le `how='left'` garde toutes les lignes de gauche et complète à NaN les colonnes de droite sans correspondance. Un junior s'attend parfois au comportement de `how='inner'`, qui aurait fait disparaître ce client silencieusement.
- 13. a)** `indicator=True` ajoute une colonne `_merge` (`left_only`, `right_only`, `both`) qui rend visible d'où vient chaque ligne. Sans lui, un junior ne détecte souvent une perte de lignes qu'après coup, en comparant les longueurs avant et après merge.
- 14. b)** `concat()` empile des DataFrames au même schéma sans logique de clé; `merge()` aligne deux tables selon une colonne commune. Utiliser `merge()` pour simplement empiler deux mois de ventes produit une explosion de lignes ou des colonnes `_x/_y` inutiles.
- 15. c)** pandas ajoute automatiquement les suffixes `_x` (gauche) et `_y` (droite) sur les colonnes homonymes non utilisées comme clé de jointure. Ne pas préciser `suffixes=...` explicitement rend le résultat illisible pour la suite de l'analyse.
- 16. d)** `iloc[2]` sélectionne la 3^e ligne par position, peu importe la valeur de l'index. `loc[3]` cherche une ligne dont l'index vaut exactement 3, ce qui n'est pas la même chose si l'index a été trié ou filtré : confusion très fréquente chez les juniors.
- 17. a)** `and/or` fonctionnent élément par élément pour des booléens Python simples, pas pour des Series pandas : il faut `&` et `|`, avec chaque condition entre parenthèses à cause de la priorité des opérateurs. C'est l'un des tout premiers plantages pandas rencontrés par un junior.
- 18. b)** `isin()` teste l'appartenance à une liste de valeurs en une seule expression lisible. Empiler des `|` fonctionne aussi, mais devient vite illisible et source d'erreur de parenthésage dès qu'il y a plus de deux ou trois valeurs.
- 19. c)** Le `SettingWithCopyWarning` apparaît car pandas ne peut pas garantir si `sub` est une vue ou une copie de `df`. La correction robuste : `.copy()` explicite si tu veux travailler sur une copie indépendante, ou directement `df.loc[condition, colonne] = valeur` sur l'original. Ignorer l'avertissement est le réflexe le plus dangereux, car la modification peut silencieusement ne rien changer.
- 20. d)** `drop(columns=[...], inplace=True)` supprime la colonne directement sur `df`. Sans `axis=1` ni `columns=`, `drop()` cherche par défaut une ligne (`axis=0`), ce qui déclenche une `KeyError` : confusion fréquente chez les juniors qui débutent avec `drop()`.

Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre analyste. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

